

Count Number of Teams

Company: Goldman Sachs

Year: Not specified

Difficulty: Medium

Topic: Arrays, Counting, Brute Force

Source: [LeetCode 1395 – Count Number of Teams](#)

Problem Description

You are given an integer array `rating`, where every soldier has a **unique rating**.

You need to choose exactly **3 soldiers** to form a team.

For three soldiers at indices:

$$i < j < k$$

the team is valid if either:

$$\text{rating}[i] < \text{rating}[j] < \text{rating}[k]$$

or:

$$\text{rating}[i] > \text{rating}[j] > \text{rating}[k]$$

In simple terms, the three ratings must be either:

- **Strictly increasing**, or
- **Strictly decreasing**

A soldier can be part of multiple teams.

Return the total number of valid teams.

Input Format

- The first line contains a single integer `N` — the number of soldiers.
- The second line contains `N` space-separated integers denoting the array `rating`.

Output Format

- Print a single integer — the total number of valid teams that can be formed.

Constraints

- $3 \leq N \leq 1000$
- $1 \leq \text{rating}[i] \leq 10^5$
- All integers in rating are unique.

Example 1

Input:

$N = 5$

rating = [2,5,3,4,1]

Output:

3

Explanation

The three valid teams are:

(2,3,4)

(5,4,1)

(5,3,1)

Therefore:

3

valid teams can be formed.

Example 2

Input:

$N = 3$

rating = [2,1,3]

Output:

0

Explanation

There is no combination of three soldiers that is completely increasing or completely decreasing.

Therefore:

0

is returned.

Example 3

Input:

$N = 4$

rating = [1,2,3,4]

Output:

4

Explanation

The valid teams are:

(1,2,3)

(1,2,4)

(1,3,4)

(2,3,4)

All four teams are strictly increasing.

Therefore:

4

is the answer.

Approach to Solve This Problem

We can solve this problem using a **counting approach with three positions**.

For every soldier j , consider it as the **middle soldier** of the team.

A valid team can be:

Increasing

$\text{rating}[i] < \text{rating}[j] < \text{rating}[k]$

For the current j , we need:

- An element on the **left** smaller than $\text{rating}[j]$.
- An element on the **right** greater than $\text{rating}[j]$.

So:

increasing teams =

leftSmaller $\times$ rightGreater

Decreasing

Similarly, for:

$\text{rating}[i] > \text{rating}[j] > \text{rating}[k]$

we need:

- An element on the **left** greater than $\text{rating}[j]$.
- An element on the **right** smaller than $\text{rating}[j]$.

So:

decreasing teams =

leftGreater $\times$ rightSmaller

For every middle position j , add both values to the answer.

Example Dry Run

Consider:

rating = [2,5,3,4,1]

Take 5 as the middle element:

Left: [2]

Right: [3,4,1]

For increasing teams:

leftSmaller = 1

rightGreater = 0

So:

$$1 \times 0 = 0$$

For decreasing teams:

leftGreater = 0

rightSmaller = 2

So:

$$0 \times 2 = 0$$

Now take 3 as the middle:

Left: [2,5]

Right: [4,1]

Increasing:

leftSmaller = 1

rightGreater = 1

$$1 \times 1 = 1$$

This gives:

(2,3,4)

Decreasing:

leftGreater = 1

rightSmaller = 1

$1 \times 1 = 1$

This gives:

(5,3,1)

We repeat this for every possible middle element and add the counts.

Java Solution

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int[] rating = new int[n];
```

```
        for (int i = 0; i < n; i++) {
```

```
            rating[i] = sc.nextInt();
```

```
}
```

```
int teams = 0;
```

```
// Consider every soldier as the middle soldier
```

```
for (int j = 0; j < n; j++) {
```

```
    int leftSmaller = 0;
```

```
    int leftGreater = 0;
```

```
    int rightSmaller = 0;
```

```
    int rightGreater = 0;
```

```
// Check elements on the left
```

```
for (int i = 0; i < j; i++) {
```

```
    if (rating[i] < rating[j]) {
```

```
        leftSmaller++;
```

```
    } else if (rating[i] > rating[j]) {
```

```
        leftGreater++;
```

```
    }
```

```
}
```

```
// Check elements on the right
```

```
for (int k = j + 1; k < n; k++) {
```

```
        if (rating[k] < rating[j]) {  
            rightSmaller++;  
        } else if (rating[k] > rating[j]) {  
            rightGreater++;  
        }  
    }  
}  
  
// Increasing teams  
teams += leftSmaller * rightGreater;  
  
// Decreasing teams  
teams += leftGreater * rightSmaller;  
}  
  
System.out.println(teams);  
}  
}
```

Solution:

1. **Read the input:** main() reads N, then reads N integers into rating, matching the Input Format.
2. **Consider each soldier as the middle soldier:** For every index j, treat rating[j] as the middle element of a possible team.
3. **Count smaller and greater elements on the left:** Traverse all indices before j. Count how many ratings are smaller than rating[j] as leftSmaller and how many are greater as leftGreater.

4. **Count smaller and greater elements on the right:** Traverse all indices after j . Count how many ratings are smaller than `rating[j]` as `rightSmaller` and how many are greater as `rightGreater`.
5. **Count increasing teams:** An increasing team requires a smaller rating on the left and a greater rating on the right. Therefore, add `leftSmaller * rightGreater` to the total number of teams.
6. **Count decreasing teams:** A decreasing team requires a greater rating on the left and a smaller rating on the right. Therefore, add `leftGreater * rightSmaller` to the total.
7. **Repeat for every middle soldier:** Every possible team has exactly one middle index j , so considering every soldier as the middle soldier counts every valid team exactly once.
8. **Print the result:** After checking all possible middle positions, print teams, matching the Output Format.

Complexity Analysis

- **Time Complexity:** $O(N^2)$ — for every middle element, we scan the elements on its left and right.
- **Space Complexity:** $O(N)$ — the input array requires $O(N)$ space; the counting variables use $O(1)$ extra space.